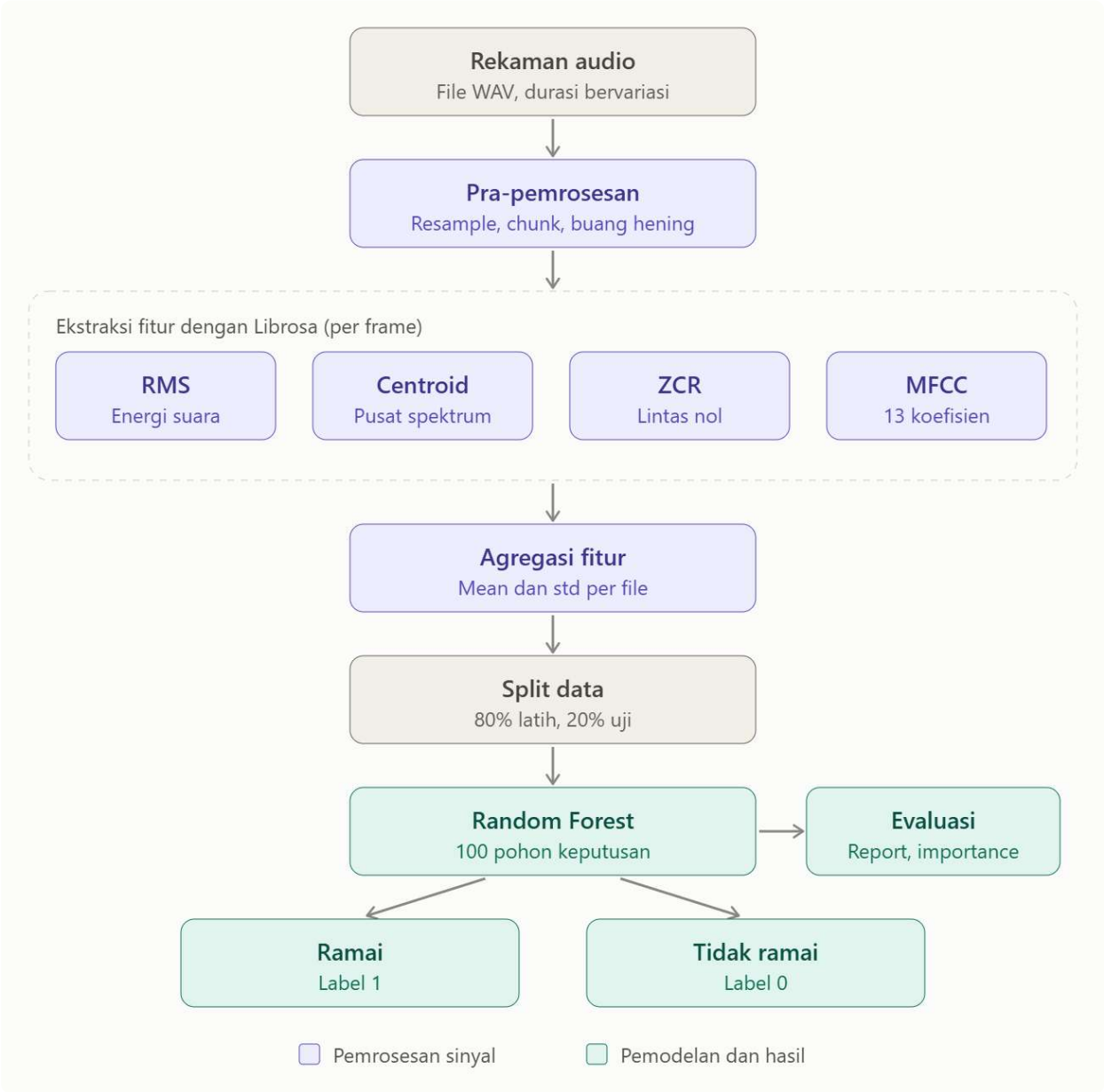
(https://hackmd.io/@ProyekSai / Klasifikasi Audio



Audio Klasifikasi



- **Pra-pemrosesan** mencakup resample ke 16 kHz, pemotongan rekaman panjang menjadi chunk 5–10 detik, dan pembuangan frame hening (tips 1 dan 3 di dokumen).
- **Ekstraksi fitur** menghasilkan deret waktu per frame, sehingga panjangnya berbeda antarfile. Karena itu perlu agregasi sebelum masuk ke Random Forest.
- **Agregasi** mengubah tiap file menjadi satu vektor statis. Pada kode di dokumen, jumlahnya 32 fitur, yaitu 2 (RMS) + 2 (centroid) + 2 (ZCR) + 13 mean MFCC + 13 std MFCC. Dokumen memakai `np.random.rand(100, 40)` sebagai data dummy, jadi angka 40 itu tidak sesuai dengan fungsi `extract_features`.

- **Split data** sebaiknya dilakukan per file asli, bukan per chunk. Kalau chunk dari satu rekaman tersebar di data latih dan uji, hasil evaluasi akan terlalu optimistis.
- **Evaluasi** memakai `classification_report` dan `feature_importances_` seperti di kode dokumen.

Klasifikasi Audio

1. Ekstraksi Fitur dengan Librosa

Untuk membedakan ruangan "ramai" (banyak orang berbicara/beraktivitas) dan "tidak ramai" (sepi atau hanya suara latar seperti AC), Anda perlu mengekstrak fitur yang merepresentasikan energi, frekuensi, dan karakteristik suara manusia.

Beberapa fitur utama yang bisa Anda ambil dari Librosa:

- **RMS (Root Mean Square):** Mengukur energi atau volume keseluruhan dari audio. Ruangan yang ramai pasti memiliki energi suara yang lebih tinggi. Rumus RMS adalah $x_{rms} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} x_i^2}$
- **Spectral Centroid:** Menunjukkan "kecerahan" atau pusat massa dari distribusi frekuensi. Suara percakapan manusia memiliki centroid yang berbeda dibandingkan suara dengung mesin (seperti kipas angin atau AC). Rumus Spectral Centroid adalah $C = \frac{\sum_{k=1}^{K} k \cdot X(k)}{\sum_{k=1}^{K} X(k)}$
- **MFCC (Mel-Frequency Cepstral Coefficients):** Merepresentasikan timbre atau bentuk pendek suara. Ini adalah fitur terbaik untuk mendeteksi keberadaan suara manusia (speech) karena meniru cara telinga manusia bekerja.
- **Zero Crossing Rate (ZCR):** Mengukur seberapa sering sinyal audio melewati nilai nol. Suara percakapan yang ramai cenderung memiliki ZCR yang lebih tinggi dibandingkan suara bising frekuensi rendah.

2. Klasifikasi dengan Random Forest

Random Forest adalah pilihan yang sangat tepat untuk kasus ini karena:

- **Tidak butuh data raksasa:** Berbeda dengan Deep Learning (seperti CNN), Random Forest bisa bekerja sangat baik dengan dataset berukuran ratusan hingga ribuan file audio.

- **Menangani Non-Linearitas:** Hubungan antara fitur audio (seperti RMS dan MFCC) dengan label "ramai" seringkali tidak linear. Random Forest menangani ini dengan baik melalui pohon keputusan berganda.
- **Feature Importance:** Model ini akan memberi tahu Anda fitur mana yang paling berpengaruh. Misalnya, Anda mungkin menemukan bahwa `mean(RMS)` dan `mean(MFCC_3)` adalah penentu utama keramaian.

3. Pipeline Implementasi

Karena file audio memiliki durasi yang berbeda-beda, Anda tidak bisa langsung memasukkan array audio ke dalam Random Forest. Anda harus melakukan **Agregasi Fitur** untuk mengubah fitur *time-series* menjadi satu vektor statis per file audio.

Vektor fitur akhir untuk satu file audio akan terlihat seperti ini:

$$F = [mean(RMS), std(RMS), mean(MFCC_1), \dots, mean(Spectral_Centroid)]$$

4. Contoh Kode Python

Berikut adalah implementasi dasar menggunakan `librosa` dan `scikit-learn`:

```
import librosa
import numpy as np
import os
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# 1. Fungsi Ekstraksi Fitur
def extract_features(file_path):
    # Load audio dengan sample rate yang konsisten
    y, sr = librosa.load(file_path, sr=16000)

    # Ekstraksi fitur
    rms = librosa.feature.rms(y=y)[0]
    cent = librosa.feature.spectral_centroid(y=y, sr=sr)[0]
    zcr = librosa.feature.zero_crossing_rate(y=y)[0]
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)

    # Agregasi fitur (mengambil mean dan standar deviasi)
    features = np.hstack([
        np.mean(rms), np.std(rms),
        np.mean(cent), np.std(cent),
        np.mean(zcr), np.std(zcr),
        np.mean(mfcc, axis=1), np.std(mfcc, axis=1)
    ])
    return features

# 2. Persiapan Data (Dummy contoh)
# Asumsikan Anda memiliki folder 'ramai' dan 'tidak_ramai'
X = []
y = []

# for file in os.listdir('path/to/ramai'):
#     X.append(extract_features(os.path.join('path/to/ramai', file)))
#     y.append(1) # Label 1 untuk ramai

# for file in os.listdir('path/to/tidak_ramai'):
#     X.append(extract_features(os.path.join('path/to/tidak_ramai', file)))
#     y.append(0) # Label 0 untuk tidak ramai

# Data dummy untuk contoh
X = np.random.rand(100, 40) # 100 sampel, 40 fitur
y = np.random.randint(0, 2, 100) # Label acak 0 atau 1

# 3. Split Data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# 4. Training Random Forest
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# 5. Evaluasi
y_pred = rf_model.predict(X_test)
```

```
print(classification_report(y_test, y_pred))

# Melihat Feature Importance
importances = rf_model.feature_importances_
print("Fitur paling penting:", np.argsort(importances)[::-1][:5])
```

Akurasi bergantung pada:

1. **Handling Silence:** Jika audio rekaman Anda memiliki banyak jeda hening, hitunglah fitur hanya pada *frame* yang memiliki energi di atas ambang batas (threshold) tertentu. Ini mencegah model belajar dari "kesunyian" yang panjang.
2. **Data Augmentation:** Jika dataset Anda sedikit, tambahkan variasi pada audio pelatihan dengan menambahkan *background noise* acak, mengubah *pitch* sedikit, atau mengubah kecepatan (time-stretching) menggunakan Librosa.
3. **Chunking:** Jika rekaman Anda sangat panjang (misal 1 jam), potong menjadi *chunk* berukuran 5-10 detik, ekstrak fitur dari setiap *chunk*, lalu beri label pada setiap *chunk* tersebut. Ini akan memperbanyak data latih secara signifikan.